



Playwright

Complete Testing Guide

From Installation to Running Tests — Step by Step

Covers: Setup · Writing Tests · Locators · Assertions · Debugging · CI/CD

Table of Contents

#	Section	Contents
1	What is Playwright?	Overview, features, browser support
2	Prerequisites	Node.js, npm, system requirements
3	Installation	npm init, install Playwright, browsers
4	Project Structure	Folder layout, config file explained
5	Write Your First Test	Page, navigate, click, fill, assert
6	Locator Strategies	getByRole, getByText, CSS, XPath
7	Core Assertions	toBeVisible, toHaveText, toHaveURL...
8	Running Tests	CLI flags, headed, debug, grep

9	Playwright Config	browsers, timeout, retries, reporters
10	Page Object Model	Reusable page classes, best practices
11	Debugging Tests	UI Mode, Inspector, traces, screenshots
12	API Testing	request fixture, REST calls in tests
13	CI/CD Integration	GitHub Actions workflow
14	Quick Reference	Cheat sheet of commands & locators

1. What is Playwright?

Playwright is a modern, open-source end-to-end testing framework by Microsoft. It lets you write tests that control real browsers — Chromium, Firefox, and WebKit — from a single API. Tests can run headlessly (no GUI) or headed, locally or in CI.

Feature	Details
Multi-browser	Chromium (Chrome/Edge), Firefox, WebKit (Safari)
Multi-language	JavaScript, TypeScript, Python, Java, C#
Auto-wait	Waits for elements automatically — no manual sleeps
Network intercept	Mock APIs, block requests, inspect traffic
Parallel tests	Run tests in parallel across workers
Trace viewer	Record & replay test execution step-by-step
Screenshots/Video	Capture on failure or always

2. Prerequisites

Before installing Playwright, make sure the following are in place on your machine:

A Node.js (v18 or higher)

Download from nodejs.org. Playwright requires Node.js 18+.

Verify after install:

```
node --version    # should print v18.x.x or higher
npm --version     # should print 9.x.x or higher
```

B A code editor

Visual Studio Code is recommended. Install the **Playwright Test for VSCode** extension for run buttons, IntelliSense, and integrated debugging.

C Terminal / Command Prompt

Use any terminal: VS Code's integrated terminal, Windows PowerShell, macOS Terminal, or Linux bash.

■ **Note:** Playwright works on Windows, macOS, and Linux. No browser installation is needed beforehand — Playwright downloads its own browser binaries.

3. Installation — Step by Step

1 Create a project folder

Open your terminal and run the commands below to create and enter a new folder.

```
mkdir my-playwright-project  
cd my-playwright-project
```

2 Initialise a Node.js project

This creates a package.json file that tracks your project's dependencies.

```
npm init -y
```

3 Install Playwright

This is the one-command setup. It installs the `@playwright/test` package, downloads Chromium, Firefox, and WebKit browser binaries, creates a `playwright.config.ts` file, and generates example tests.

```
npm init playwright@latest
```

4 Answer the setup prompts

The wizard asks a few questions:

- **TypeScript or JavaScript?** → Choose TypeScript (recommended) or JavaScript
- **Tests folder name?** → Press Enter to accept default (tests/)
- **GitHub Actions workflow?** → y if you use GitHub CI
- **Install browsers?** → y (required on first install)

5 Verify the installation

Run the example tests that Playwright created:

```
npx playwright test
```

■ **Tip:** After a successful install you will see a summary like: '3 passed (3 tests)' plus a line showing the HTML report path.

4. Project Structure

After running the wizard, your project folder looks like this:

```
my-playwright-project/
├── tests/                ← your test files (.spec.ts)
│   └── example.spec.ts  ← auto-generated sample
├── tests-examples/      ← more advanced examples
├── playwright.config.ts  ← main config file
├── package.json          ← project metadata & scripts
├── package-lock.json
└── node_modules/        ← installed packages
```

playwright.config.ts — Key Sections

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',      // folder with test files
  fullyParallel: true,     // run tests in parallel
  retries: 1,              // retry failed tests once
  workers: 4,              // parallel workers
  reporter: 'html',        // generate HTML report
  use: {
    baseURL: 'http://localhost:3000', // base URL
    trace: 'on-first-retry', // record trace on retry
    screenshot: 'only-on-failure', // capture on failure
    video: 'retain-on-failure', // keep video on failure
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ]
});
```

```
1,  
});
```

5. Write Your First Test

Playwright test files use the **.spec.ts** (or **.spec.js**) naming convention. Create a new file at **tests/login.spec.ts**:

Full Test File — Login Flow

```
import { test, expect } from '@playwright/test';

// 'test.describe' groups related tests
test.describe('Login Page', () => {

  // Runs before each test in this block
  test.beforeEach(async ({ page }) => {

    await page.goto('https://example.com/login');

  });

  test('should display the login form', async ({ page }) => {

    // Assert heading is visible
    await expect(page.getByRole('heading', { name: 'Sign In' }))
      .toBeVisible();

  });

  test('should login with valid credentials', async ({ page }) => {

    // Fill the email field
    await page.getByLabel('Email').fill('user@example.com');

    // Fill the password field
    await page.getByLabel('Password').fill('mypassword123');

    // Click the submit button
    await page.getByRole('button', { name: 'Sign In' }).click();

    // Assert we are redirected to the dashboard
    await expect(page).toHaveURL('/dashboard');

    await expect(page.getByText('Welcome back!')).toBeVisible();

  });

});
```



```
});

test('should show error for wrong password', async ({ page }) => {

  await page.getByLabel('Email').fill('user@example.com');

  await page.getByLabel('Password').fill('wrongpassword');

  await page.getByRole('button', { name: 'Sign In' }).click();

  await expect(page.getByText('Invalid credentials')).toBeVisible();

});

});
```

Key Concepts in This Test

Concept	Explanation
test(name, fn)	Defines one test case. fn receives fixtures like { page }.
async/await	All Playwright actions are async. Always use await.
page.goto(url)	Navigates the browser to a URL.
page.getByLabel()	Finds an input by its associated <label> text — robust locator.
page.fill(value)	Types a value into an input field (clears first).
page.click()	Clicks an element. Playwright auto-waits for it to be actionable.
expect(locator)	Starts an assertion. Playwright retries until timeout.
test.beforeEach	Hook that runs setup code before every test in the describe block.

6. Locator Strategies

Locators are how Playwright finds elements on the page. Prefer user-facing locators (role, label, text) over CSS/XPath — they are more resilient.

Recommended Locators (use these first)

```
// By ARIA role – most resilient, mirrors how users interact
page.getByRole('button', { name: 'Submit' })
page.getByRole('link', { name: 'Home' })
page.getByRole('textbox', { name: 'Search' })

// By label text – great for form inputs
page.getByLabel('Email address')
page.getByLabel('Password')

// By placeholder text
page.getByPlaceholder('Enter your email')

// By visible text
page.getByText('Welcome back!')
page.getByText('Submit', { exact: true })

// By test id (data-testid attribute) – best for dynamic content
page.getByTestId('submit-btn') // <button data-testid='submit-btn'>

// By alt text – for images
page.getByAltText('Company logo')

// By title attribute
page.getByTitle('Close dialog')
```

CSS & XPath (use as fallback)

```
// CSS selector
page.locator('.login-form button[type=submit]')
page.locator('#email-input')
```

```
// XPath

page.locator('xpath=//button[contains(text(),"Sign In")]')

// Chaining locators

page.locator('.modal').getByRole('button', { name: 'Confirm' })

// Filter by another locator

page.getByRole('listitem').filter({ hasText: 'Product A' })
```

■ **Tip:** Use `npx playwright codegen https://yoursite.com` to auto-generate locators by clicking on elements in the browser.

7. Core Assertions

Playwright uses **web-first assertions** that automatically retry until the condition is met or the timeout is reached. Always use **await** with `expect()`.

```
// ■■ Visibility ████████████████████████████████████████  
  
await expect(locator).toBeVisible();  
  
await expect(locator).toBeHidden();  
  
  
// ■■ Text content ████████████████████████████████████████  
  
await expect(locator).toHaveText('exact text');  
  
await expect(locator).toContainText('partial text');  
  
await expect(locator).toHaveText(/regex pattern/);  
  
  
// ■■ Input values ████████████████████████████████████████  
  
await expect(locator).toHaveValue('input value');  
  
await expect(locator).toBeChecked(); // checkbox  
  
await expect(locator).not.toBeChecked();  
  
await expect(locator).toBeEnabled();  
  
await expect(locator).toBeDisabled();  
  
  
// ■■ Page-level ████████████████████████████████████████  
  
await expect(page).toHaveURL('https://example.com/dashboard');  
  
await expect(page).toHaveURL(/dashboard/);  
  
await expect(page).toHaveTitle('My App - Dashboard');  
  
  
// ■■ Count & attributes ████████████████████████████████████  
  
await expect(locator).toHaveCount(5);  
  
await expect(locator).toHaveAttribute('href', '/home');  
  
await expect(locator).toHaveClass('active');  
  
  
// ■■ Negation — add .not. before any matcher ████████████  
  
await expect(locator).not.toBeVisible();  
  
await expect(page).not.toHaveURL('/error');
```

```
// ■■ Custom timeout ████████████████████████████████████████████████████████
```

```
await expect(locator).toBeVisible({ timeout: 10000 }); // 10s
```

8. Running Tests

Command	CLI Flag / Usage
Run all tests	<code>npx playwright test</code>
Run a specific file	<code>npx playwright test tests/login.spec.ts</code>
Run tests matching a name	<code>npx playwright test --grep "login"</code>
Run in headed mode (see browser)	<code>npx playwright test --headed</code>
Run with UI Mode (interactive)	<code>npx playwright test --ui</code>
Run on one browser only	<code>npx playwright test --project=chromium</code>
Run in debug mode (step-by-step)	<code>npx playwright test --debug</code>
Run with Playwright Inspector	<code>PWDEBUG=1 npx playwright test</code>
Show the last HTML report	<code>npx playwright show-report</code>
Run tests in parallel (4 workers)	<code>npx playwright test --workers=4</code>
Retry failed tests 3 times	<code>npx playwright test --retries=3</code>
Generate code by clicking (Codegen)	<code>npx playwright codegen https://example.com</code>
List all available tests	<code>npx playwright test --list</code>

add scripts to package.json

```
"scripts": {  
  "test": "playwright test",  
  "test:headed": "playwright test --headed",  
  "test:debug": "playwright test --debug",  
  "test:ui": "playwright test --ui",  
  "report": "playwright show-report"  
}  
  
// Then just run: npm test
```

9. Playwright Config Deep Dive

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  // ■■ Test discovery ████████████████████████████████████████████

  testDir:    './tests',

  testMatch:   '**/*.spec.{ts,js}',    // pattern for test files


  // ■■ Execution ████████████████████████████████████████████████████████████

  fullyParallel: true,

  workers: process.env.CI ? 2 : 4,      // fewer workers in CI

  retries:    process.env.CI ? 2 : 0,    // retry only in CI


  // ■■ Reporting ████████████████████████████████████████████████████████████

  reporter: [

    ['html', { open: 'never' }],

    ['list'],    // prints each test result in terminal

  ],


  // ■■ Shared browser settings ████████████████████████████████████████████████████████████

  use: {

    baseURL:     'http://localhost:3000',

    headless:    true,

    viewport:    { width: 1280, height: 720 },

    actionTimeout: 10_000,    // per action (click, fill...)

    navigationTimeout: 30_000,    // page.goto()

    trace:       'on-first-retry',

    screenshot:  'only-on-failure',

    video:       'retain-on-failure',

    locale:      'en-US',

    timezoneId:  'America/New_York',

  },

});
```

[illegible]

10. Page Object Model (POM)

The Page Object Model keeps tests clean by encapsulating locators and actions in separate classes. When the UI changes, you update one class, not every test.

pages/LoginPage.ts — Page class

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {

  readonly page: Page;

  readonly emailInput: Locator;

  readonly passInput: Locator;

  readonly submitBtn: Locator;

  readonly errorMsg: Locator;

  constructor(page: Page) {

    this.page = page;

    this.emailInput = page.getByLabel('Email');

    this.passInput = page.getByLabel('Password');

    this.submitBtn = page.getByRole('button', { name: 'Sign In' });

    this.errorMsg = page.getByTestId('error-message');

  }

  async goto() {

    await this.page.goto('/login');

  }

  async login(email: string, password: string) {

    await this.emailInput.fill(email);

    await this.passInput.fill(password);

    await this.submitBtn.click();

  }

}
```

tests/login.spec.ts — Using the page class

```
import { test, expect } from '@playwright/test';

import { LoginPage } from '../pages/LoginPage';

test('valid login redirects to dashboard', async ({ page }) => {

  const loginPage = new LoginPage(page);

  await loginPage.goto();

  await loginPage.login('user@example.com', 'password123');

  await expect(page).toHaveURL('/dashboard');

});

test('wrong password shows error', async ({ page }) => {

  const loginPage = new LoginPage(page);

  await loginPage.goto();

  await loginPage.login('user@example.com', 'wrongpass');

  await expect(loginPage.errorMessage).toContainText('Invalid credentials');

});
```

11. Debugging Tests

UI Mode — best starting point

Launches an interactive GUI showing all tests, a browser preview, and timeline.

```
npx playwright test --ui
```

Debug Mode — step through actions

Pauses before each Playwright action. You control execution with the DevTools console.

```
npx playwright test --debug  
npx playwright test tests/login.spec.ts --debug
```

Playwright Inspector (PWDEBUG)

Opens the Inspector panel alongside the browser for action-by-action stepping.

```
PWDEBUG=1 npx playwright test          # Linux/Mac  
set PWDEBUG=1 && npx playwright test   # Windows CMD
```

page.pause() — pause inside test

Add this line inside your test to pause at that exact point.

```
await page.pause(); // add anywhere in your test
```

Screenshots on failure

Playwright auto-saves screenshots when screenshot: 'only-on-failure' is in config.

Manual screenshot:

```
await page.screenshot({ path: 'debug-screenshot.png' });
```

Trace Viewer — record & replay

Traces capture a full recording of actions, network, screenshots, and DOM snapshots.

```
npx playwright test --trace on  
npx playwright show-trace trace.zip
```

12. API Testing with Playwright

Playwright can test REST APIs directly using the **request** fixture — no browser needed. You can also combine API calls with UI tests.

Pure API test

```
import { test, expect } from '@playwright/test';

test('GET /api/users returns 200', async ({ request }) => {

  const response = await request.get('https://api.example.com/users');

  expect(response.status()).toBe(200);

  const body = await response.json();

  expect(body).toHaveProperty('users');

  expect(body.users.length).toBeGreaterThan(0);

});

test('POST /api/users creates a user', async ({ request }) => {

  const response = await request.post('https://api.example.com/users', {

    data: { name: 'Alice', email: 'alice@example.com' },

    headers: { 'Authorization': 'Bearer token123' },

  });

  expect(response.status()).toBe(201);

  const user = await response.json();

  expect(user.name).toBe('Alice');

});
```

API login then UI test (hybrid)

```
test('login via API then test UI', async ({ page, request }) => {

  // 1. Get auth token from API

  const res = await request.post('/api/auth/login', {

    data: { email: 'user@example.com', password: 'pass' }
```

```
});  
  
const { token } = await res.json();  
  
// 2. Set cookie/localStorage to skip UI login  
await page.goto('/');  
await page.evaluate(t => localStorage.setItem('token', t), token);  
  
// 3. Navigate to protected page directly  
await page.goto('/dashboard');  
await expect(page.getByText('Welcome')).toBeVisible();  
});
```

13. CI/CD — GitHub Actions

Create a file at `.github/workflows/playwright.yml` in your repository:

```
name: Playwright Tests

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      - name: Run Playwright tests
        run: npx playwright test

      - name: Upload HTML report
```

```
uses: actions/upload-artifact@v4

if: always() # upload even if tests fail

with:

  name: playwright-report

  path: playwright-report/

  retention-days: 14
```

■ **Note:** The `--with-deps` flag installs all OS-level browser dependencies needed on Ubuntu (fonts, libraries). Always include this in CI.

14. Quick Reference Cheat Sheet

Essential CLI Commands

Action	Command
Install	<code>npm init playwright@latest</code>
Run all tests	<code>npx playwright test</code>
Headed mode	<code>npx playwright test --headed</code>
Single file	<code>npx playwright test file.spec.ts</code>
Filter by name	<code>npx playwright test --grep "keyword"</code>
Debug mode	<code>npx playwright test --debug</code>
UI mode	<code>npx playwright test --ui</code>
Codegen	<code>npx playwright codegen https://site.com</code>
Show report	<code>npx playwright show-report</code>
Install browsers	<code>npx playwright install</code>

Locator Cheat Sheet

Locator	Usage
<code>getByRole(role, {name})</code>	Button, link, heading, textbox, etc.
<code>getByLabel(text)</code>	Form inputs with a <label>
<code>getByPlaceholder(text)</code>	Inputs with placeholder attribute
<code>getByText(text)</code>	Any element containing text
<code>getByTestId(id)</code>	data-testid attribute (CI-safe)
<code>getByAltText(text)</code>	Images by alt attribute
<code>locator(css)</code>	CSS selector (fallback)
<code>locator('xpath=...')</code>	XPath (last resort)

Official Docs: playwright.dev | VS Code Extension: [Playwright Test for VSCode](#) | GitHub: [microsoft/playwright](https://github.com/microsoft/playwright)